Week 7 — Building Production ETL Pipelines with PySpark & Delta Lake ACID Transactions

Candidate Name:	Himanshu Batra
Track / Domain:	Data Engineering Internship (Week-7 Deliverable)
Organization:	Celebal Technologies
University	DIT University
Date of Submission:	3rd August 2026

1. Executive Abstract

Modern big data platforms require high-throughput data processing engines coupled with storage format layers that support ACID transactions, fine-grained row-level mutation, and automated schema enforcement. Traditional data lakes based on raw CSV or un-indexed Parquet files suffer from data corruption, lack of transactional safety, and inability to perform cost-effective row updates. This report documents the design, architecture, and implementation of an enterprise-grade Data Engineering pipeline processing the US Superstore retail dataset. Utilizing Apache Spark (PySpark) and Delta Lake 3.2, the pipeline achieves data hygiene, feature engineering, and automated incremental updates via Delta MERGE INTO (UPSERT) operations.

2. Problem Statement & Objectives

Retail analytics engines require reliable daily ingestion of transactional data containing updates to existing orders and new customer purchases. Overwriting entire datasets daily is computationally inefficient and prone to failure. The objective of this project is to build a production pipeline that:

- Ingests raw, unstructured transactional logs (10,800 records).
- Executes comprehensive data cleaning, null imputation, and string standardization.
- Engineers 9 business-oriented financial and operational features.
- Implements Delta Lake table storage to enable ACID compliance, schema enforcement, and time travel.
- Demonstrates Delta Lake MERGE (UPSERT) logic to seamlessly combine incoming updates with existing target tables.

3. Technology Stack & System Architecture

The pipeline leverages an industry-standard modern data stack:

- Processing Framework: Apache Spark 3.5.1 (PySpark)
- Storage Layer: Delta Lake 3.2.0 (Parquet columnar format + _delta_log JSON metadata commits)
- Data Analytics & Visualization: Pandas 2.0+, Seaborn 0.12+, Matplotlib 3.7+
- Environment: Databricks Notebook / Local PySpark runtime

4. Data Cleaning & Feature Engineering Methodology

Data hygiene was enforced across multiple stages before loading into Delta tables. Duplicates were eliminated, column names were normalized to standard snake_case, string fields were trimmed and converted to InitCap, and date fields were explicitly cast to DateType.

Derived Business Features

- profit_margin: (profit / sales) * 100 — Identifies product profitability tier.
- revenue_category: High (>= $1000), Medium (>= $500), Low (>= $100), Micro (< $100).

- discount_bucket: No Discount, Low (1-20%), Medium (21-40%), Heavy (>40%).
- shipping_delay: Datediff between Order Date and Ship Date.
- business_segment: Star Performer, Profitable, Break-Even, Loss-Making.

5. Delta Lake MERGE (UPSERT) Deep Dive

Delta Lake MERGE INTO evaluates ON match conditions between a target table and incoming source DataFrame. When matched, specified columns (sales, profit, quantity) are updated. When unmatched, new records are inserted.

In our execution test, the base target table contained 500 rows. An incoming batch contained 50 price updates and 50 new orders. The MERGE operation executed atomically, resulting in exactly 550 rows in the target Delta table without requiring a full table rewrite.

6. Executive Business Insights & Visualizations

Based on statistical analysis of 9,994 cleaned transactions:

- Regional Leadership: The West region generated the highest total revenue ($725,457), followed by East ($678,781).
- Category Performance: Technology is the top revenue generator ($836,154) with highest average profit margins.
- Discount Cannibalization: Discounts exceeding 20% heavily degrade profit margins, resulting in severe losses in Furniture (Chairs, Tables).

7. Engineering Challenges & Solutions

- Schema Enforcement: Enforced strict column datatypes during ingestion to avoid runtime Delta merge errors.
- Concurrency & Transaction Logs: Observed how Delta's _delta_log JSON files maintain serializable snapshot isolation.

8. Conclusion & References

This project validates the efficacy of Delta Lake for enterprise data pipelines, ensuring data reliability, high query speed, and low operational overhead for incremental ETL workloads.

- Delta Lake Documentation: https://learn.microsoft.com/en-us/azure/databricks/delta/merge
- Superstore Dataset: Kaggle Final Dataset